Семинар 4. Коммуникация и транспорт: REST, RPC, Queues и др.

Распределенные вычисления

Георгий Семенов

Институт прикладных компьютерных наук
Университет ИТМО

весна 2026

Организационные моменты

- Первое домашнее задание – жесткий дедлайн в 23:59 19.03.2026
- Транспортным протоколам будет посвящена вторая домашка (?)

Содержание

1. Зачем нужен транспорт?
2. Античность: CORBA и EJB
3. Барокко: REST и SOAP
4. Классицизм: gRPC, Thrift, Java RMI, ...
5. Модернизм: очереди сообщений
6. Немного про тестирование распределенных систем

Что мы будем понимать под транспортом?

- Транспорт – это способ обмена данными между компонентами распределенной системы
  - Обеспечивает коммуникацию, абстрагируя от деталей сети и инфраструктуры
- Характеристики транспортных протоколов
  - Формат сообщений (схема, (де-)сериализация)
  - Управление сессией (handshakes, heartbeats, timeouts)
  - Гарантии доставки (exactly-once, at-least-once, at-most-once)
- Какой транспорт бывает?
  - REST API (HTTP); RPC (gRPC, Thrift, RPyC, JSON-RPC, XML-RPC, Java RMI, ...)
  - Message Queues (RabbitMQ, Kafka, ...)

Где транспорт в OSI?

Это Session (5) и Application (7) уровни OSI модели. Transport (4) – это про TCP и UDP, а мы говорим про более высокоуровневые протоколы.

OSI (Open Source Interconnection) 7 Layer Model

Layer	Application/Example		Central Device/ Protocols		DOD4 Model
Application (7) Serves as the window for users and application processes to access the network services.	**End User layer** Program that opens what was sent or creates what is to be sent Resource sharing • Remote file access • Remote printer access • Directory services • Network management		**User Applications** SMTP	GATEWAY Can be used on all layers	Process
Presentation (6) Formats the data to be presented to the Application layer. It can be viewed as the "Translator" for the network.	**Syntax layer** encrypt & decrypt (if needed) Character code translation • Data conversion • Data compression • Data encryption • **Character Set Translation**		JPEG/ASCII EBDIC/TIFF/GIF PICT		
Session (5) Allows session establishment between processes running on different stations.	**Synch & send to ports** (logical ports) Session establishment, maintenance and termination • Session support - perform security, name recognition, logging, etc.		**Logical Ports** RPC/SQL/NFS NetBIOS names		
Transport (4) Ensures that messages are delivered error-free, in sequence, and with no losses or duplications.	**TCP** Host to Host, Flow Control Message segmentation • Message acknowledgement • Message traffic control • Session multiplexing	PACKET FILTERING	TCP/SPX/UDP		Host to Host
Network (3) Controls the operations of the subnet, deciding which physical path the data takes.	**Packets** ("letter", contains IP address) Routing • Subnet traffic control • Frame fragmentation • Logical-physical address mapping • Subnet usage accounting		**Routers** IP/IPX/ICMP		Internet
Data Link (2) Provides error-free transfer of data frames from one node to another over the Physical layer.	**Frames** ("envelopes", contains MAC address) [NIC card —— Switch —— NIC card] (end to end) Establishes & terminates the logical link between nodes • Frame traffic control • Frame sequencing • Frame acknowledgment • Frame delimiting • Frame error checking • Media access control		**Switch Bridge WAP** PPP/SLIP	Land	

OSI: уровни 1–3

- **L1 — Physical (физический)**
  - Передача битов по физической среде (кабель, радиоволны, оптика)
  - Примеры: Ethernet PHY, Wi-Fi (802.11) физика, USB
  - Понятий «адресации» нет — только сигналы
- **L2 — Data Link (канальный)**
  - Передача кадров (frames) между узлами в пределах одного сегмента сети
  - Адресация: MAC-адреса (48 бит)
  - Примеры: Ethernet (802.3), Wi-Fi (802.11), PPP, VLAN
  - Контроль ошибок и коллизий (CSMA/CD, CSMA/CA)
- **L3 — Network (сетевой)**
  - Маршрутизация пакетов между сетями
  - Адресация: IP-адреса (IPv4/IPv6)
  - Примеры: IP, ICMP (`ping`), протоколы маршрутизации (OSPF, BGP)
  - Не гарантирует доставку, порядок, отсутствие дубликатов

Время вычислять по IP

Вопрос: можно ли определить наше местоположение по IP-адресу?

OSI L4 — Transport (транспортный уровень)

Задача: обеспечить логический канал между процессами (не машинами).

- Адресация: **порты** (0–65535) идентифицируют процесс на хосте
- Два главных протокола: **TCP** и **UDP**

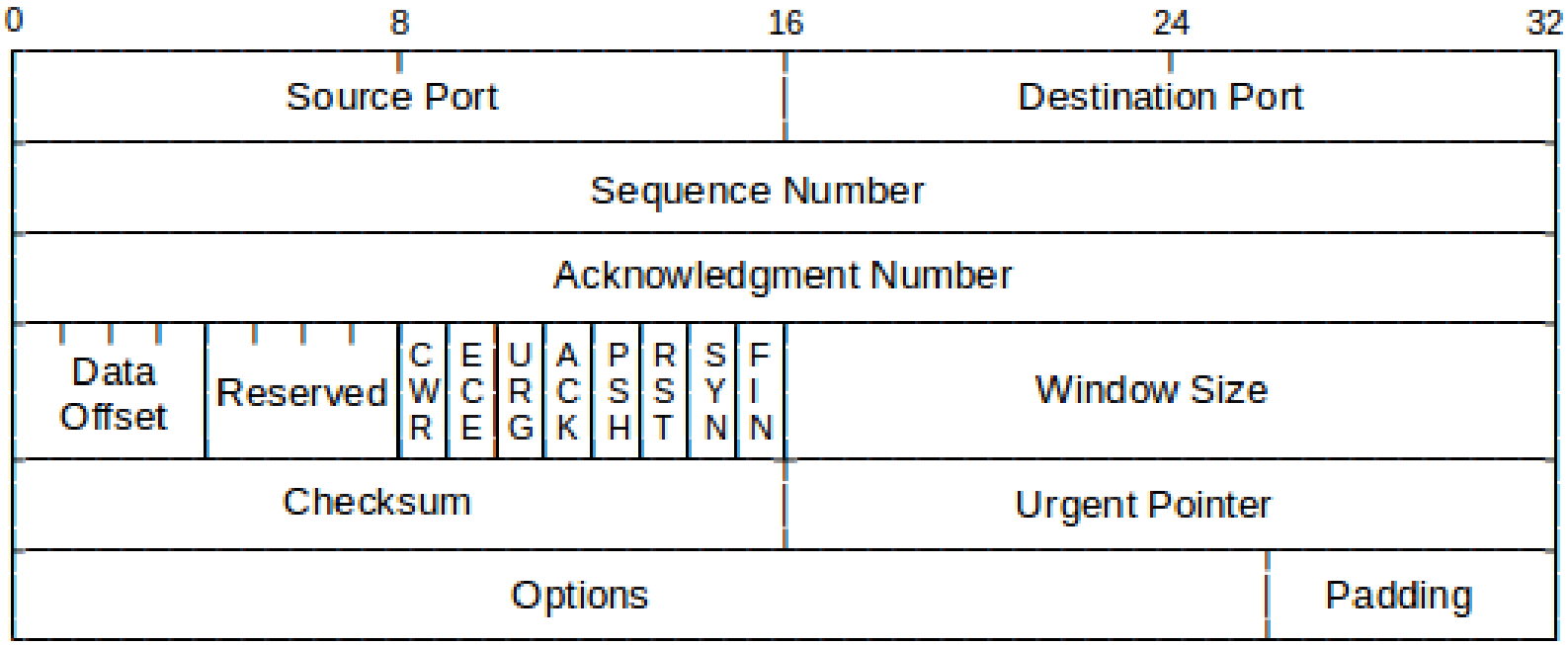
Свойство	**TCP**	**UDP**
Установка соединения	Да (3-way handshake)	Нет
Гарантия доставки	Да (ACK + retransmit)	Нет
Порядок пакетов	Да	Нет
Контроль потока	Да (sliding window)	Нет
Overhead заголовка	20+ байт	8 байт
Задержка	Выше	Ниже

TCP — как устроено надёжное соединение

- **3-way handshake**: SYN → SYN-ACK → ACK (установка соединения)
- **4-way teardown**: FIN → ACK → FIN → ACK (закрытие)
- **Sequence numbers**: каждый байт пронумерован; получатель отправляет ACK
- **Retransmission**: при отсутствии ACK — повторная отправка (timeout / fast retransmit)
- **Sliding window**: отправитель посылает несколько сегментов без ожидания ACK
- **Congestion control**: алгоритмы (CUBIC, BBR) предотвращают перегрузку сети

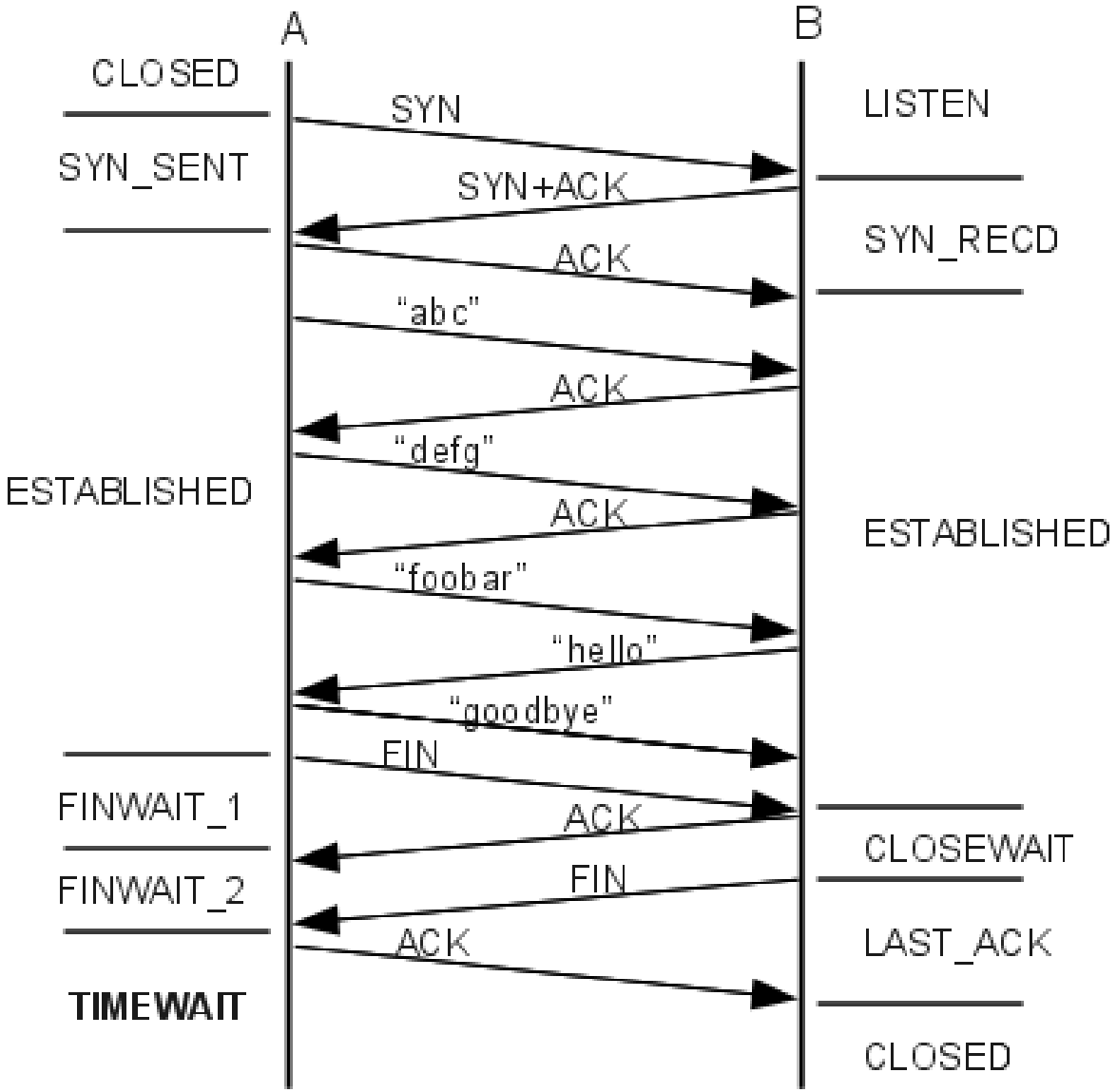
Следствие для распределённых систем: TCP скрывает потери, но не скрывает **задержки** и не защищает от **head-of-line blocking** (пакет потерян — вся очередь ждёт).

Заголовок TCP-пакета

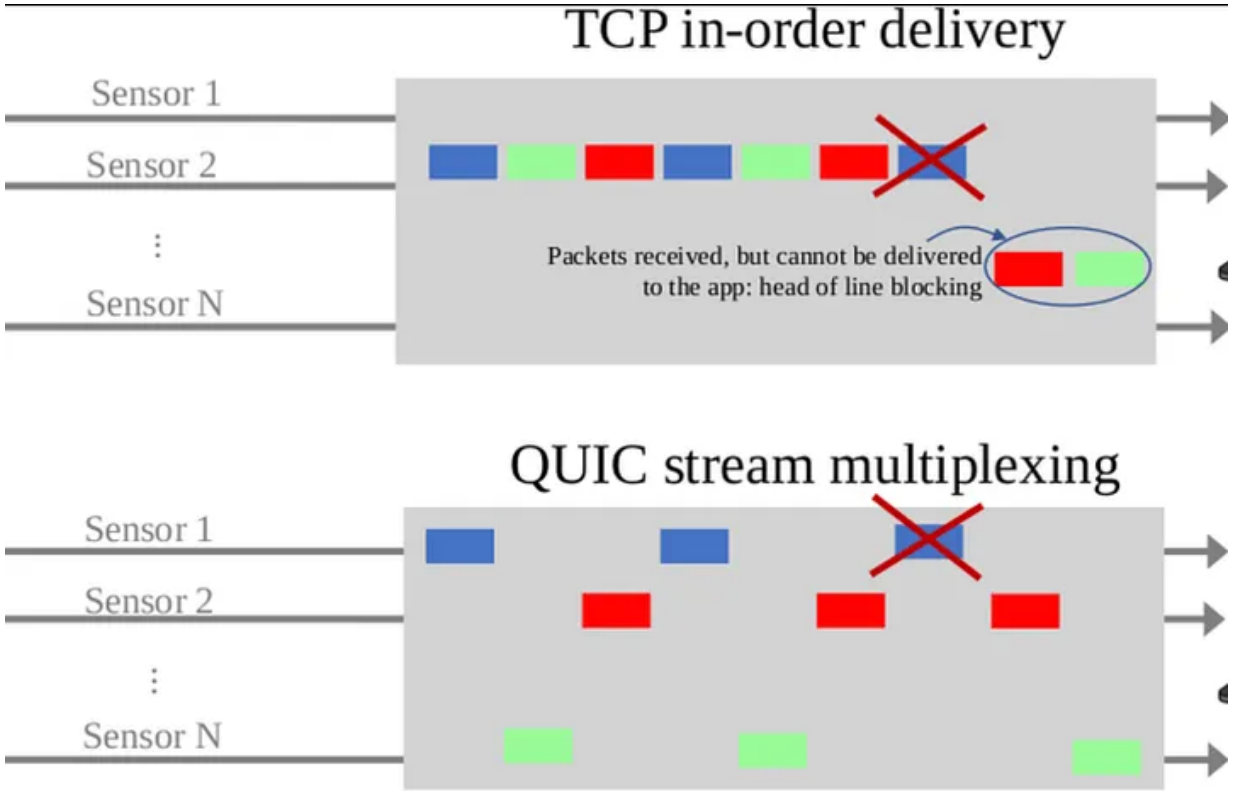


- **Source/Destination Port** — идентификаторы процессов на отправителе и получателе
- **Sequence Number** — номер первого байта в сегменте (база для упорядочивания и повторной отправки)
- **Acknowledgment Number** — подтверждение: `ACK = N` означает, что получены все байты до `N-1`
- **Flags** (SYN, ACK, FIN, RST, PSH, URG) — управление жизненным циклом соединения
- **Window Size** — сколько байт получатель готов принять (flow control)
- **Checksum** — контроль целостности заголовка и данных

Handshake & Teardown



Head-of-line blocking



UDP — когда отказываются от гарантий

- Нет соединения, нет ACK, нет порядка — *fire-and-forget*
- **Где это уместно:**
  - DNS (один short запрос/ответ, retransmit на application level)
  - Стриминг видео/аудио (устаревший пакет бесполезен)
  - Онлайн-игры (позиция игрока — нужна свежая, а не старая)
  - QUIC (HTTP/3) — реализует надёжность поверх UDP для устранения head-of-line blocking TCP
- **Главное преимущество:** минимальная задержка и управление на уровне приложения
- Multicast и broadcast возможны только через UDP

Заголовок UDP-пакета

0	16	32
Source Port	Destination Port	
Length	Data Checksum	

- **Source/Destination Port** — порты приложений отправителя и получателя
- **Length** — длина UDP-датаграммы в байтах (заголовок + payload)
- **Checksum** — контроль целостности заголовка и данных (в IPv6 обязателен)
- Заголовок очень маленький (8 байт): нет sequence number, ACK, окна, флагов
- Поэтому UDP быстрый и простой, но надёжность/порядок при необходимости реализуются на уровне приложения

Содержание

1. Зачем нужен транспорт?
2. Античность: CORBA и EJB
3. Барокко: REST и SOAP
4. Классицизм: gRPC, Thrift, Java RMI, ...
5. Модернизм: очереди сообщений
6. Немного про тестирование распределенных систем

Античность на примерах: где это реально жило

- **Банки:** карточный процессинг, платёжный шлюз, anti-fraud как отдельные удалённые сервисы
- **Телеком:** биллинг, CRM, provisioning, сеть IN через enterprise bus
- **Гос-системы:** тяжёлые интеграции между ведомствами через централизованные middleware

Типичный стек конца 90-х / начала 2000-х

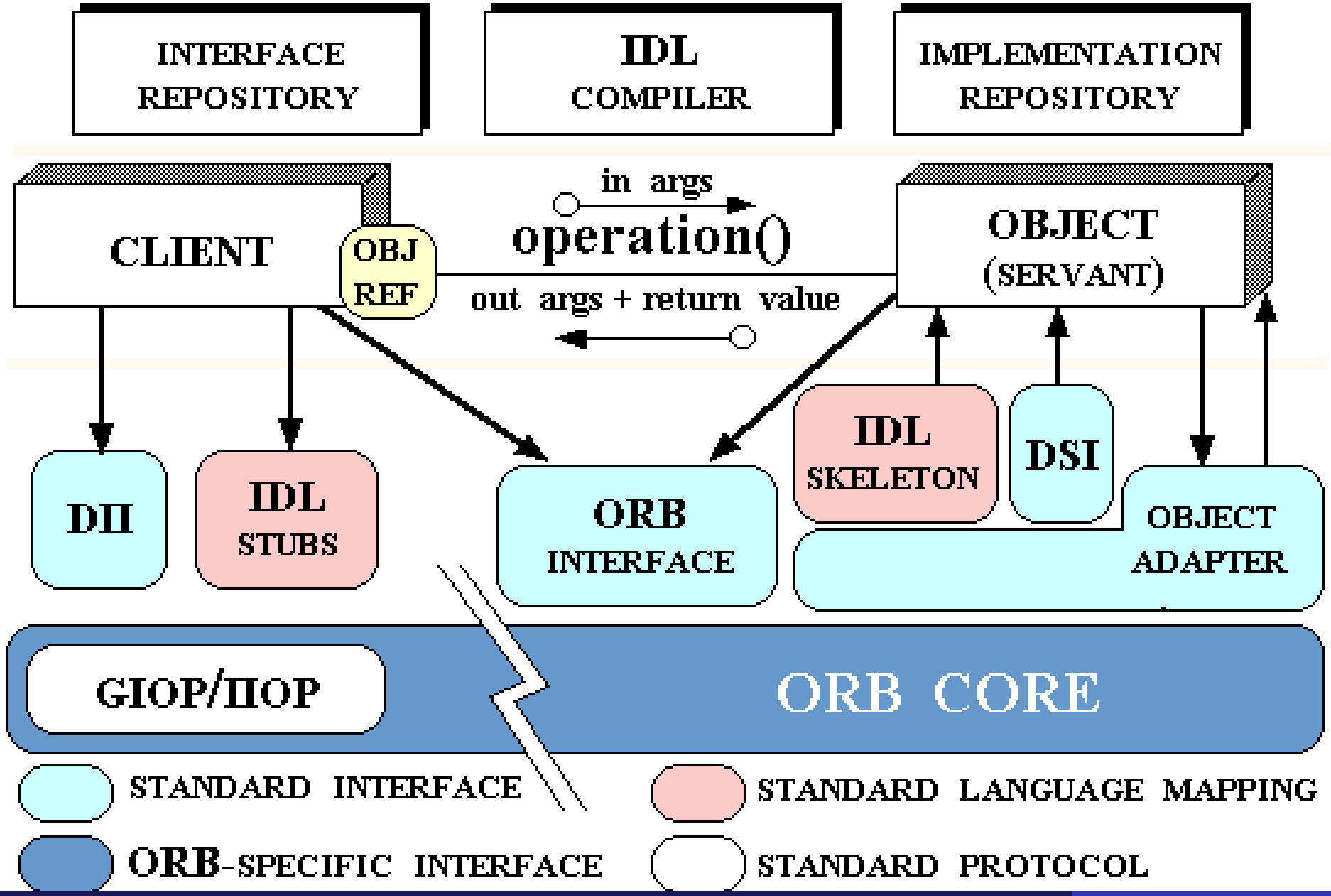
App Server (WebLogic/WebSphere/JBoss) + ORB/CORBA + EJB + Oracle DB + JMS.

CORBA: пример IDL-контракта

```
module bank {
  interface PaymentService {
    string pay(in string accountId,
               in double amount,
               in string currency);
  };
};
```

- Из IDL генерировались стабы/скелетоны для C++/Java
- Клиент видел вызов как обычный метод, но под капотом шёл IIOP
- Контракт строгий: несовместимые изменения ломают клиентов

Ох уж эта CORBA



CORBA: как выглядел вызов

```
org.omg.CORBA.Object ref =
    orb.string_to_object(ior);
PaymentService svc =
    PaymentServiceHelper.narrow(ref);

String txId = svc.pay("ACC-42", 199.99, "RUB");
```

- `narrow(...)` — приведение к нужному удалённому интерфейсу
- IOR/Naming Service — как клиент находит удалённый объект
- Ошибки часто проявлялись как таймауты ORB и проблемы совместимости версий

EJB: минимальный пример бизнес-бина

```
@Stateless
public class PaymentBean {
  @PersistenceContext EntityManager em;

  @TransactionAttribute(REQUIRED)
  public String pay(String accountId, BigDecimal amount) {
    // списание, запись транзакции, аудит
    return UUID.randomUUID().toString();
  }
}
```

- Транзакция открывается контейнером автоматически
- Без ручного JDBC begin/commit/rollback в бизнес-коде

EJB: асинхронная обработка через MDB

```
@MessageDriven(activationConfig = {
  @ActivationConfigProperty(
    propertyName = "destination",
    propertyValue = "jms/PaymentsQueue")
})
public class FraudCheckMDB implements MessageListener {
  public void onMessage(Message msg) {
    // anti-fraud проверка транзакции
  }
}
```

- Практический пайплайн: API → JMS Queue → MDB worker
- Ретраи, dead-letter и pooling часто настраивались на уровне контейнера

Античные паттерны в современном виде

Тогда	Сейчас
CORBA IDL + IIOP	gRPC + protobuf + HTTP/2
EJB Stateless Bean	Spring Service / Micronaut Bean
EJB TransactionAttribute	@Transactional в современных фреймворка
Message-Driven Bean + JMS	Kafka/RabbitMQ consumer
App Server clustering	Kubernetes + service mesh

Вывод: подходы не исчезли, они переродились в более лёгких и web-native инструментах.

Содержание

1. Зачем нужен транспорт?
2. Античность: CORBA и EJB
3. Барокко: REST и SOAP
4. Классицизм: gRPC, Thrift, Java RMI, ...
5. Модернизм: очереди сообщений
6. Немного про тестирование распределенных систем

REST — Representational State Transfer

- REST — архитектурный стиль для распределённых гипермедиа-систем, описан Roy Fielding в диссертации (2000)
- Не протокол, а набор **архитектурных ограничений** поверх HTTP

Ключевые понятия

- **Ресурс** — любая именованная сущность (пользователь, заказ, документ)
- **Представление** — сериализованное состояние ресурса: JSON, XML, ...
- **URI** — унифицированный идентификатор ресурса (`/users/42`)

Принципы REST

1. **Client–Server** — разделение ответственности: клиент управляет UI, сервер — данными
2. **Stateless** — каждый запрос содержит всю необходимую информацию; сессионное состояние хранится на клиенте
3. **Cacheable** — ответы явно указывают, можно ли их кэшировать
4. **Uniform Interface** — единый интерфейс: идентификация ресурсов через URI, манипуляция через представления, HATEOAS
5. **Layered System** — клиент не знает, общается ли он с конечным узлом или промежуточным (балансировщик, кэш)
6. **Code on Demand** (опционально) — сервер может передавать исполняемый код

HTTP методы

Метод	Idempotent	Safe	Использование
`GET`	Да	Да	Получить ресурс
`HEAD`	Да	Да	Получить только заголовки
`POST`	Нет	Нет	Создать ресурс / вызов действия
`PUT`	Да	Нет	Заменить ресурс целиком
`PATCH`	Нет	Нет	Частично обновить ресурс
`DELETE`	Да	Нет	Удалить ресурс

Safe — не меняет состояние сервера
Idempotent — повторный запрос даёт тот же результат

HTTP статус-коды

- **2xx — Success**: `200 OK`, `201 Created`, `204 No Content`
- **3xx — Redirection**: `301 Moved Permanently`, `304 Not Modified`
- **4xx — Client Error**:
  - `400 Bad Request` — неверный формат запроса
  - `401 Unauthorized` — требуется аутентификация
  - `403 Forbidden` — нет прав доступа
  - `404 Not Found` — ресурс не найден
  - `409 Conflict` — конфликт состояния (напр. дубликат)
  - `429 Too Many Requests` — rate limiting
- **5xx — Server Error**: `500 Internal Server Error`, `502 Bad Gateway`, `503 Service Unavailable`

REST — форматы данных

Чаще всего используется JSON, реже XML или другие форматы.

```
{
  "id": 42,
  "name": "Alice",
  "email": "alice@example.com",
  "roles": ["admin", "user"]
}
```

```
<user>
  <id>42</id>
  <name>Alice</name>
  <email>
    alice@example.com
  </email>
</user>
```

Content negotiation: клиент указывает предпочтительный формат через заголовки `Accept` и `Content-Type`.

REST — поддержание совместимости

Как не сломать клиентов при изменении API?

- **URL-версионирование** — `/api/v1/users`, `/api/v2/users`
  - Просто и явно, но засоряет URL-пространство
- **Header-версионирование** — `Accept: application/vnd.api+json;version=2`
  - Чище по духу REST, но сложнее для клиентов и кэшей
- **Эволюция схемы без сломки (non-breaking changes):**
  - Добавлять новые поля можно — клиенты игнорируют неизвестные поля
  - Новые поля должны быть опциональными
  - Не удалять и не переименовывать существующие поля
  - Помечать deprecated-поля и поддерживать несколько версий до удаления

SOA и SOAP

SOA — Service-Oriented Architecture

Архитектурный подход, при котором функциональность системы разбивается на сервисы, взаимодействующие через формально определённые интерфейсы.

SOAP (Simple Object Access Protocol) — протокол обмена сообщениями на основе XML, типичный транспорт SOA-систем.

- Разработан Microsoft, IBM и др. (1998), стандарт W3C
- Поддерживает разные транспорты: HTTP, SMTP, JMS, TCP
- Строгая типизация и контракт через **WSDL**
- Встроенные WS-стандарты: безопасность (WS-Security), транзакции (WS-AtomicTransaction), надёжная доставка (WS-ReliableMessaging)

SOAP — структура сообщения

```
<soap:Envelope
    xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
    xmlns:m="http://example.org/stock">
  <soap:Header>
    <m:Auth>
      <m:token>secret-token-123</m:token>
    </m:Auth>
  </soap:Header>
  <soap:Body>
    <m:GetStockPrice>
      <m:StockName>AAPL</m:StockName>
    </m:GetStockPrice>
  </soap:Body>
</soap:Envelope>
```

- **Envelope** — обёртка, определяет содержимое как SOAP-сообщение
- **Header** — метаданные: аутентификация, транзакции, маршрутизация
- **Body** — основная полезная нагрузка (payload); **Fault** — описание ошибки

WSDL — Web Services Description Language

WSDL — XML-язык описания интерфейса веб-сервиса (аналог header-файла в C).

```
<definitions name="StockQuote"
    targetNamespace="http://example.org/stock">
  <message name="GetPriceInput">
    <part name="symbol" type="xsd:string"/>
  </message>
  <message name="GetPriceOutput">
    <part name="price"  type="xsd:float"/>
  </message>
  <portType name="StockQuotePortType">
    <operation name="GetPrice">
      <input  message="GetPriceInput"/>
      <output message="GetPriceOutput"/>
    </operation>
  </portType>
</definitions>
```

Совместимость: изменение WSDL требует новой версии контракта; клиенты перегенерируют стабы (`wsdl2java`, `wsimport`).

REST vs SOAP

Критерий	REST	SOAP
Транспорт	HTTP	HTTP, SMTP, TCP, ...
Формат данных	JSON, XML, ...	XML (только)
Контракт	OpenAPI / Swagger	WSDL (строгий)
Безопасность	HTTPS, OAuth 2.0	WS-Security
Производительность	Выше	Ниже
Типизация	Слабая	Строгая
Применение	Публичные API, web	Корпоративный сектор

REST — для большинства новых проектов. SOAP — когда нужны встроенные стандарты надёжности, безопасности и транзакционности.

Содержание

1. Зачем нужен транспорт?
2. Античность: CORBA и EJB
3. Барокко: REST и SOAP
4. Классицизм: gRPC, Thrift, Java RMI, ...
5. Модернизм: очереди сообщений
6. Немного про тестирование распределенных систем

Remote Procedure Call (RPC)

Идея

Вызов удалённой функции должен выглядеть как вызов локальной функции.

- **Stub / Proxy** — локальный объект, перехватывающий вызов
- **Marshalling** — преобразование аргументов в байты для передачи
- **Transport** — доставка сообщения по сети
- **Unmarshalling** — восстановление аргументов на стороне сервера

Фундаментальная проблема RPC — частичные отказы:

- Запрос дошёл, ответ — нет: выполнилось ли?
- Ответ пришёл с задержкой: нужна идемпотентность
- Сервер недоступен: нужны retry, circuit breaker, timeout

gRPC — определение и .proto

- **gRPC** — RPC-фреймворк от Google (2015); использует **Protocol Buffers** для сериализации и **HTTP/2** в качестве транспорта
- Поддерживает: C++, Java, Go, Python, Rust, Node.js, C#, Ruby, ...

```
syntax = "proto3";
service UserService {
  rpc GetUser   (GetUserRequest)   returns (User);
  rpc ListUsers (ListUsersRequest) returns (stream User);
}
message GetUserRequest { int64 id = 1; }
message User {
  int64  id    = 1;
  string name  = 2;
  string email = 3;
}
```

gRPC — forward/backward совместимость

Protocol Buffers обеспечивает совместимость через **номера полей** (field numbers).

- **Backward compatibility** — новый сервер понимает старых клиентов:
  - Можно добавлять новые поля; старые клиенты их не заполняют — сервер видит значения по умолчанию
- **Forward compatibility** — старый сервер понимает новых клиентов:
  - Неизвестные поля игнорируются (proto3)
- **Запрещено:** удалять или переиспользовать **номера полей**; менять тип поля
- **Рекомендуется:** при удалении поля помечать его номер как `reserved`

Итог: клиент и сервер можно обновлять независимо (independent deployability).

gRPC — производительность и HTTP/2

- **Бинарная сериализация** (protobuf) — в 3–10× компактнее JSON
- **HTTP/2 мультиплексирование** — множество запросов в одном TCP-соединении без head-of-line blocking
- **Header compression** (HPACK) — экономия трафика
- **Стриминг** — 4 режима взаимодействия:
  - Unary (1 запрос → 1 ответ, аналог обычного HTTP)
  - Server streaming (1 запрос → поток ответов)
  - Client streaming (поток запросов → 1 ответ)
  - Bidirectional streaming (поток ↔ поток)
- Встроены: deadlines/timeouts, cancellation, metadata, interceptors

Apache Thrift

- Разработан в Facebook (2007), передан в Apache Foundation
- Генерирует код из **IDL** (.thrift) для 25+ языков: Java, C++, Python, Go, Ruby, Erlang, Haskell, ...

```
namespace java org.example

struct User {
  1: i64    id,
  2: string name,
  3: optional string email
}

service UserService {
  User       getUser  (1: i64 id),
  list<User> listUsers()
}
```

Thrift — протоколы и транспорты

Thrift разделяет **протокол** (сериализацию) и **транспорт** независимо:

Протоколы:

- `BinaryProtocol` — компактный бинарный
- `CompactProtocol` — variable-length encoding, ещё меньше
- `JSONProtocol` — JSON для отладки и веб-интеграции

Транспорты:

- `TSocket` — TCP
- `THttpClient` — HTTP
- `TMemoryBuffer` — in-memory (тесты)
- `TFramedTransport` — обёртка для async

Совместимость: аналогично protobuf — номера полей обеспечивают совместимость; `optional` поля можно добавлять без сломки клиентов.

gRPC vs Thrift

Критерий	gRPC	Thrift
Основатель	Google	Facebook (Apache)
Транспорт	HTTP/2 (только)	TCP, HTTP, memory, ...
Сериализация	protobuf (только)	binary, compact, JSON
Поддержка языков	≈12	≈28
Стриминг	Встроенный (4 вида)	Нет
Популярность сейчас	Выше	Ниже

Когда Thrift? Если нужна поддержка экзотических языков или гибкость транспорта, либо работаете в экосистеме, где Thrift уже используется (HBase, Scribe и др.).

Java RMI

Java Remote Method Invocation — встроенный механизм RPC для Java (JDK 1.1, 1997).

- Позволяет вызывать методы объектов на другой JVM как локальные
- Архитектура:
  - **Remote Interface** — интерфейс, расширяющий `java.rmi.Remote`
  - **Stub** — заглушка на стороне клиента (генерируется автоматически)
  - **RMI Registry** — сервис имён для поиска объектов (`rmiregistry`)
- Передаёт аргументы через **Java Serialization**
- Ограничения:
  - Только Java
  - Медленная и небезопасная десериализация (CVE)
  - Проблемы с firewall (динамические порты)
  - Практически вытеснен gRPC/Thrift в современных системах

Java RMI — пример

```java
// 1. Remote Interface
public interface Hello extends Remote {
    String sayHello() throws RemoteException;
}

// 2. Server: publish the object
HelloImpl obj = new HelloImpl();
Hello stub = (Hello) UnicastRemoteObject
        .exportObject(obj, 0);
Registry registry = LocateRegistry.getRegistry();
registry.bind("Hello", stub);

// 3. Client: lookup and invoke
Registry registry =
        LocateRegistry.getRegistry("host");
Hello stub = (Hello) registry.lookup("Hello");
String response = stub.sayHello(); // remote call!
```

Содержание

1. Зачем нужен транспорт?
2. Античность: CORBA и EJB
3. Барокко: REST и SOAP
4. Классицизм: gRPC, Thrift, Java RMI, ...
5. Модернизм: очереди сообщений
6. Немного про тестирование распределенных систем

Что такое очередь сообщений?

Message Queue

Посредник между продюсерами и консюмерами, который принимает сообщения, временно хранит их и доставляет получателям.

- **Decoupling по времени**: отправитель и получатель не обязаны быть онлайн одновременно
- **Decoupling по нагрузке**: очередь сглаживает пики входящего трафика
- **Async processing**: медленные задачи выносятся в фон
- **Fan-out**: одно сообщение может быть доставлено нескольким подписчикам

Типичный pipeline: API принимает запрос → кладёт задачу в очередь → worker обрабатывает → пишет результат в БД.

Семантики доставки

- **At-most-once**
  - Сообщение доставляется не более одного раза
  - Потери возможны, дубликатов почти нет
- **At-least-once**
  - Сообщение будет доставлено, но может прийти повторно
  - Требуется **idempotent** обработчик
- **Exactly-once** (в узком контексте)
  - Сложно и дорого в распределённых системах
  - Обычно реализуется как комбинация идемпотентности, транзакций и deduplication

Kafka — что это?

Apache Kafka — distributed commit log для потоковой передачи событий.

- Изначально создан в LinkedIn (2011), сейчас проект Apache
- Основная абстракция — **topic**, разбитый на **partitions**
- Сообщения в партиции упорядочены по offset
- Продюсеры пишут в topic, консюмеры читают независимо
- Данные реплицируются между брокерами для отказоустойчивости

Ментальная модель

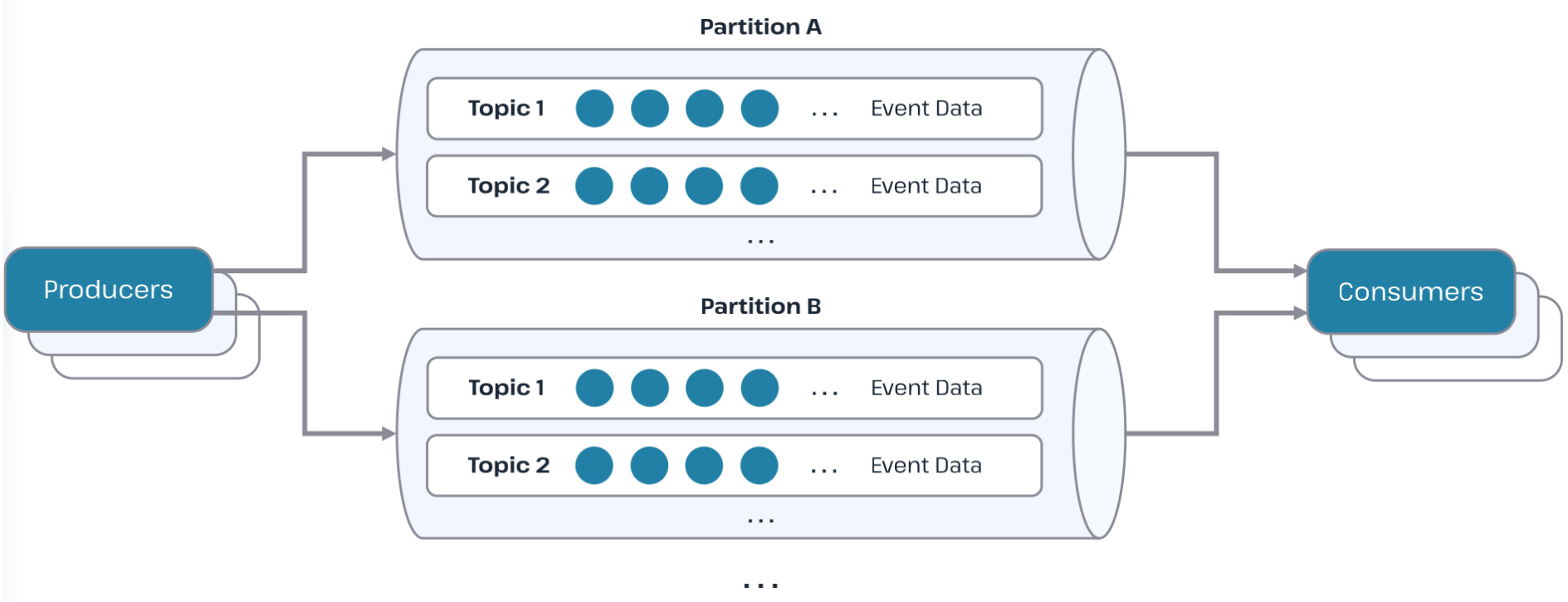
Kafka — это не классическая FIFO-очередь, а **журнал событий**, где сообщения читаются как лог.

Kafka — архитектура и масштабирование

- **Broker** — сервер Kafka, хранящий партиции
- **Topic** — логическая категория событий
- **Partition** — физический shard внутри topic
- **Replication factor** — количество копий каждой партиции
- **Leader/Follower** — запись и чтение обычно идут через лидера партиции
- **Consumer group** — масштабирование чтения: каждая партиция читается только одним консюмером в группе

Эффект: throughput растёт при увеличении числа партиций и брокеров.

Схема Kafka


Partition A
Topic 1
...
Event Data
Topic 2
...
Event Data
...
Producers
Consumers
Partition B
Topic 1
...
Event Data
Topic 2
...
Event Data
...
...

Kafka — гарантии и порядок

- Порядок сообщений гарантируется **внутри одной partition**
- Между разными partition общий порядок не гарантируется
- Обычно используется **at-least-once** доставка
- Для строгой обработки полезны:
  - idempotent producer (без дубликатов при retry)
  - transactional producer/consumer (EOS в рамках Kafka pipeline)
  - ручное управление commit offset
- Retention-based хранение: сообщения живут по времени или объёму, а не до подтверждения потребителем

Kafka — где особенно хороша

- Event sourcing и event-driven архитектуры
- Сбор логов и телеметрии (ELK, observability)
- Потоковая обработка (Kafka Streams, Flink, Spark)
- Интеграция микросервисов через immutable events

Плюсы:

- Очень высокий throughput и горизонтальное масштабирование
- Долговременное хранение и повторное чтение истории

Минусы:

- Более высокий порог входа в эксплуатацию
- Не лучшая модель для сложной маршрутизации единичных задач

RabbitMQ — что это?

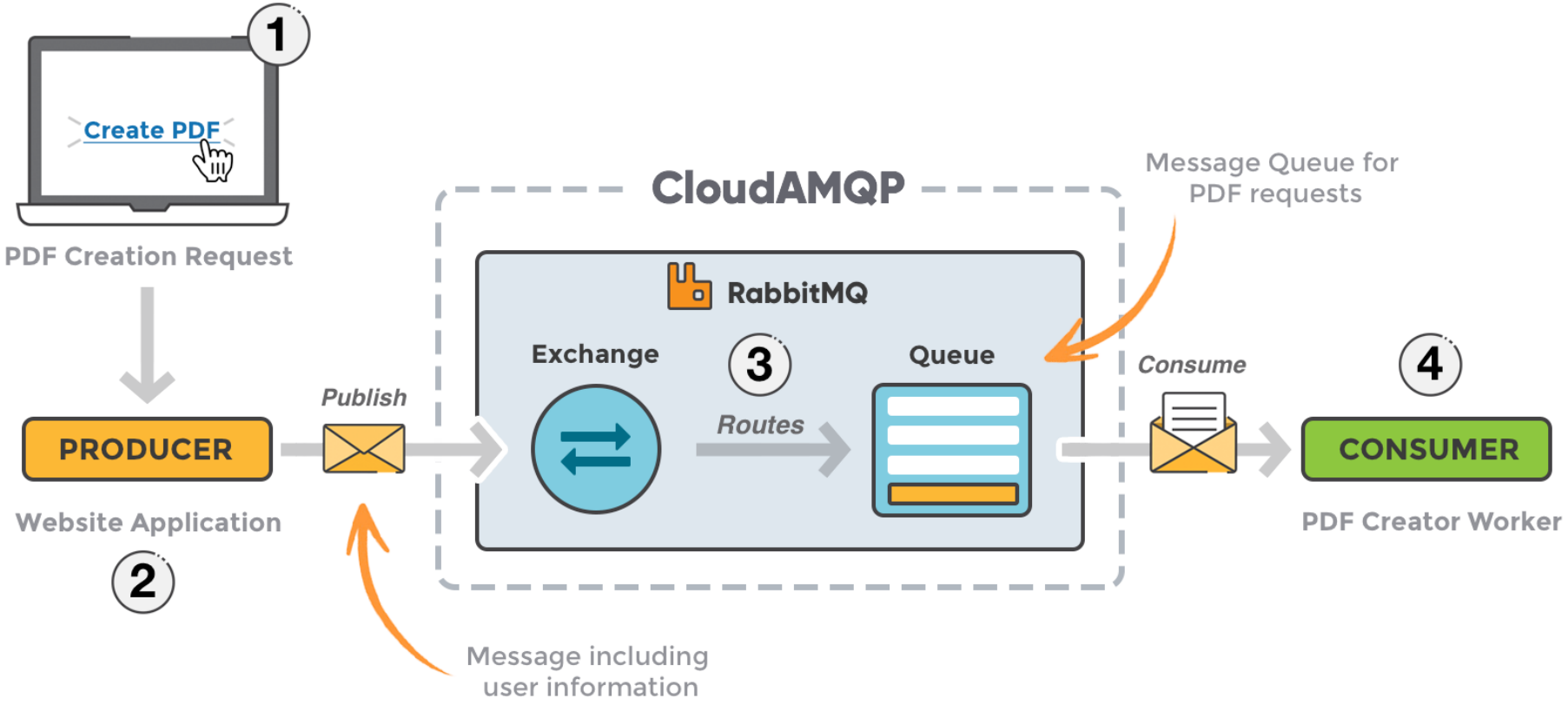
RabbitMQ — брокер сообщений на базе протокола AMQP (классическая message queue).

- Ядро написано на Erlang (высокая надёжность и конкурентность)
- Основные сущности: **exchange**, **queue**, **binding**
- Продюсер публикует сообщение в exchange
- Exchange маршрутизирует сообщение в очереди по правилам binding
- Консюмеры получают сообщения из очередей и отправляют `ack`

RabbitMQ — маршрутизация и паттерны

- **Типы exchange:**
  - `direct` — по точному routing key
  - `topic` — по шаблону (`order.*`, `*.created`)
  - `fanout` — широковещательная рассылка всем связанным очередям
  - `headers` — по заголовкам сообщения
- **Популярные паттерны:**
  - Work queue (распределение задач по воркерам)
  - Pub/Sub (fanout exchange)
  - RPC over MQ (request-reply)
  - Dead-letter queue (DLQ) для неуспешных сообщений

Схема RabbitMQ



RabbitMQ — надёжность и эксплуатация

- **Ack/Nack** — явное подтверждение обработки сообщения
- **Durable queue + persistent messages** — переживают перезапуск брокера
- **Prefetch (QoS)** — ограничивает число сообщений на консюмера
- **TTL** и **DLX** — политика повторов и отбраковки
- **Quorum queues** — современная отказоустойчивая реализация очередей

Сильная сторона RabbitMQ: гибкая маршрутизация и удобство для task queue сценариев.

Kafka vs RabbitMQ

Критерий	Kafka	RabbitMQ
Модель	Distributed log	Message broker / queues
Хранение	Retention (дни/недели)	Обычно до доставки
Порядок	Внутри partition	± Внутри очереди
Маршрутизация	Topic/partition key	Exchange + bindings
Throughput	Очень высокий	± Ниже
Основной use-case	Event streaming	Task queues, интеграции, R

Практика: часто используют вместе: RabbitMQ — для команд, интеграций и задач, Kafka — для событий и аналитики.

Содержание

1. Зачем нужен транспорт?
2. Античность: CORBA и EJB
3. Барокко: REST и SOAP
4. Классицизм: gRPC, Thrift, Java RMI, ...
5. Модернизм: очереди сообщений
6. Немного про тестирование распределенных систем

Зачем тестировать распределённые системы?

- Распределённые системы отказывают не так, как монолиты: **частичные отказы**, **гонки**, **сетевые разбиения**
- Классические unit-тесты не воспроизводят сетевые аномалии
- Баги могут проявляться только под нагрузкой или при конкретном порядке событий

Знаменитые аварии

- **Amazon DynamoDB (2012)** — потеря данных из-за ошибки в логике репликации
- **Cloudflare (2019)** — каскадный отказ из-за ошибки в firewall-правилах
- **CockroachDB (2020)** — нарушение serializability при concurrent транзакциях (найдено Jepsen)

Jepsen

- Создан Kyle Kingsbury (aphyr) в 2013 году, написан на Clojure
- Тестирует **согласованность** (consistency) и **доступность** распределённых баз данных
- **Принцип работы:**
  1. Запускает кластер в Docker-контейнерах
  2. Параллельно выполняет операции чтения/записи через клиентов
  3. Одновременно вносит **немезиды** (nemeses): network partitions, clock skew, kill nodes
  4. Собирает полную историю операций
  5. Проверяет историю на соответствие модели через алгоритмы **Elle** (serializability) и **Knossos** (linearizability)
- Нашёл критические баги в: Redis, Cassandra, ZooKeeper, MongoDB, CockroachDB, TiDB, HashiCorp Vault, ...

Chaos Engineering

Chaos Engineering

Дисциплина проверки стойкости системы путём намеренного внесения отказов в production.

- Принципы (Netflix):
  1. Определить гипотезу о поведении системы в *steady state*
  2. Намеренно вносить отказы: убивать узлы, вводить задержки, ошибки
  3. Наблюдать **отклонения** от steady state
  4. Повторять с возрастающим *blast radius*
- Инструменты:
  - **Chaos Monkey** / **Gorilla** / **Kong** (Netflix) — убивают инстансы / AZ / регион
  - **Gremlin** — коммерческая платформа для chaos experiments
  - **Chaos Mesh** — для Kubernetes: network delay, pod kill, IO fault
  - **toxiproxy** (Shopify) — proxy для эмуляции сетевых проблем

Нагрузочное и стресс-тестирование

- **Load testing** — поведение под **ожидаемой** нагрузкой
  - Метрики: latency (p50/p95/p99), throughput (RPS), error rate
- **Stress testing** — ищем поведение при **превышении** нормальной нагрузки
  - Деградирует ли система *gracefully*?
- **Soak testing** — длительная нагрузка (утечки памяти, деградация соединений)
- **Spike testing** — резкий всплеск нагрузки

Инструменты:

- **k6** (Grafana) — JavaScript DSL, удобен для CI/CD
- **Locust** — Python DSL
- **Apache JMeter** — GUI, широко используется в enterprise
- **wrk** / **wrk2** — минималистичный HTTP-бенчмаркинг

Фаззинг и формальные методы

- **Fuzzing** — генерация случайных/мутированных входов для обнаружения падений и уязвимостей
  - Применяется к: парсерам протоколов, сериализаторам, сетевым стекам
  - Инструменты: AFL++, libFuzzer, go-fuzz, jazzer (JVM)
- **Property-based testing** — проверяем инварианты на случайно сгенерированных данных
  - Инструменты: QuickCheck (Haskell), Hypothesis (Python), jqwik (Java)
- **Model checking (формальная верификация):**
  - **TLA+** (Lamport) — используется в AWS S3, Azure Cosmos DB, FoundationDB
  - **P language** (Microsoft) — проверка распределённых протоколов

Итоги

- Вспомнили про стек OSI и сетевые аспекты взаимодействия
- Поговорили про способы организации взаимодействия между компонентами: REST, SOAP, gRPC, Thrift, RabbitMQ, Kafka
- gRPC, Docker (?), Thrift (?) будет посвящена вторая домашка (?)